



מהנדסים טאלנטיים בתכנה

מבוא לתכנות מערכות

תרגיל בית מספר 2

מועד פרסום: 10.5.2026 בשעה 7:00

מועד הגשה: 28.5.2026 עד לשעה 23:59

מתרגל אחראי לתרגיל: **דוד עבדת** davidavdat@shenkar.ac.il

1 הערות כלליות

- שימו לב! לא יינתנו דחיות במועד הגשת התרגיל. תכננו את זמנכם בהתאם
- בכל שאלה בנוגע לתרגיל הבית יש לפנות לדוד. מומלץ ורצוי לפנות בקבוצת הדיסקורד. **בפניות באמצעות דוא"ל יש לכתוב בשורת הנושא (Subject) תכנות 2 תרגיל 2**
- מומלץ לקרוא ולהבין את כל ההנחיות לתרגיל לפני תחילת הפתרון.
- הגשת התרגיל תתבצע בזוגות!.
- התרגיל מנוסח בלשון זכר אך מכון לכל המינים.

2 נושא התרגיל – המשך פיתוח מערכת תחבורה ציבורית

3 מטרת התרגיל

תכנון וכתובת רשימות מקושרות כחלק מתוכנה, עבודה עם מצביעים והקצאות זיכרון דינמיות

4 חלק א (10%)

על מנת לייעל את עבודתה, ולהמנע מבאגים דומים לאלו שהיו, חילקה דניאלה את קוד התכנית שלה לפונקציות נפרדות.

להלן מובאת פונקציה להעסקת מופע יחיד של כלי תחבורה שכתבה דניאלה.

הגדרת המבנה המתאים מובאת אף היא

```
typedef struct transit_t {
    int id;
    char *type;
    int rating;
}transit;

transit *copy_transit(transit *original)
{
    result = (transit*) malloc(sizeof(transit));
    if (result == NULL) {
        return NULL;
    }
    result->type = (char *)malloc (sizeof(char) * (strlen(original->type) + 1))
    if (result->type == NULL)
    {
        free(result);
        return NULL;
    }

    strcpy(result->type, original->type);
    *result = *original;
    return result;
}

// Dany-Ella II <3
```

לאחר מספר הרצות מוצלחות על מספר קלטים שונים, הגיעה דניאלה למסקנה כי הקוד שכתבה עובד נכון משום שהתוצאות שקיבלה תאמו לציפיותיה.

4.1 סעיף א

מצאו קלט עבורו הפונקציה של דניאלה נכשלת. הסבירו בקצרה מדוע היא נכשלת והציעו תיקון פשוט כך שהפונקציה תעבוד גם עם קלט זה. (אין צורך לכתוב את הפונקציה מחדש, אלא רק אילו שינויים יש לבצע והיכן).

4.2 סעיף ב

איזו בעיה נוספת מופיעה בקוד? הסבירו בקצרה והציעו תיקונים פשוטים הפותרים את הבעיה

5 חלק ב (90%)

5.1 תיאור המערכת

לאחר ההצלחה המסחררת של הגרסה הראשונה, מעוניין משרד התחבורה להחליף את כלל מערכות ניהול התחבורה בישראל בגרסה מתקדמת ובה אין חסם על מספר התחנות והקווים אותם ניתן להוסיף למערכת.

5.2 הפעלת התוכנית

התכנית תופעל באמצעות פונקציית `main` ובה יבוצעו קריאות לפונקציות הממשק השונות על פי הפירוט בהמשך. שימו לב! את פונקציית ה-`main` יכתבו בודקי התרגילים לכן אין להגיש פונקציה כזו יחד עם המימוש שלכם. כן כדאי לכתוב פונקציית `main` כזאת לבדיקות העצמיות שלכם על מנת לוודא שהמערכת עובדת כנדרש ולבדוק את הקוד שלכם באופן מיטבי.

5.3 טיפול בשגיאות

- במקרה של שגיאה יש לדווח על סוג השגיאה המתאים בעזרת הפונקציה `prog2_report_error_message`.
- אם מופיעות כמה שגיאות יש לדווח על השגיאה החשובה ביותר. סדר החשיבות מוגדר בהגדרת הטיפוס `transport_error` הנמצא בקובץ `prog2_ex2.h`.
 - במקרה של כשלון בהקצאת זיכרון, **בכל הפקודות** יש לדווח על השגיאה `OUT_OF_MEMORY`, **לשחרר כל המשאבים שהוקצו במהלך ריצת התוכנית** ולצאת מהתוכנית.
 - במקרה שאחד או יותר מהפרמטרים שהועברו לפונקציות השונות הוא `NULL`, הפונקציה תחזיר `NULL_ARGUMENT`.
 - בכל מקרה בו הפונקציה הצליחה יש להחזיר `SUCCESS`.

5.4 פקודות

על המערכת לממש את הפונקציות הבאות

5.4.1 יצירת מערכת תחבורה חדשה

```
TransportDB *TransportCreate(void)
```

תיאור הפעולה: יצירת מבנה נתונים חדש לאחסון נתוני קווי התחבורה הציבורית
פרמטרים:
- אין

ערכי החזרה אפשריים:

- מצביע למבנה נתונים חדש ומאותחל מסוג `TransportDB` אותו אתם צריכים להגדיר
- `NULL` במקרה של כשלון

5.4.2 שחרור מערכת התחבורה

```
Void TransportDestroy(TransportDB *tdb)
```

תיאור הפעולה: מחיקת מבנה הנתונים ושחרור כל משאבי המערכת שהוקצו בזמן ריצת התכנית
פרמטרים:

- <tdb> מצביע למבנה הנתונים

ערכי החזרה אפשריים:

- אין

5.4.3 הוספת קו תחבורה למערכת

```
TransportResult TransportAddLine(TransportDB* tdb, const char *type, int  
line_id, float price)
```

תיאור הפעולה: פעולה זו מוסיפה קו תחבורה ציבורית למאגר הנתונים
פרמטרים:

- <tdb> מצביע למבנה הנתונים

- <type> סוג הקו אותו נרצה להוסיף. יכול להיות אחד מהבאים בלבד, "BUS", "METRO", "TRAIN". כל מחרוזת אחרת אינה חוקית!

- <line_id> מזהה ייחודי של הקו. מספר גדול ממש מ-0

- <price> תעריף נסיעה בודדת בקו. מספר עשרוני גדול ממש מ-0

ערכי החזרה אפשריים:

- [TRANSPORT_INVALID_LINE_TYPE](#) – במקרה בו סוג הקו אינו תקין

- [TRANSPORT_INVALID_LINE_NUMBER](#) – במקרה בו מספר הקו אינו תקין

- [TRANSPORT_ALREADY_EXISTS](#) – במקרה בו מספר הקו כבר קיים במערכת

- [TRANSPORT_INVALID_PRICE](#) – במקרה בו התעריף אינו תקין

5.4.4 הסרת קו תחבורה מהמערכת

```
TransportResult TransportRemoveLine(TransportDB* tdb, int line_id)
```

תיאור הפעולה: פעולה זו מסירה קו תחבורה ציבורית ממאגר הנתונים
פרמטרים:

- <tdb> מצביע למבנה הנתונים

- <line_id> מזהה ייחודי של הקו. מספר גדול ממש מ-0

ערכי החזרה אפשריים:

- [TRANSPORT_INVALID_LINE_NUMBER](#) – במקרה בו מספר הקו אינו תקין

- [TRANSPORT_DOESNT_EXIST](#) – במקרה בו מספר הקו אינו קיים במערכת

5.4.5 הוספת תחנת עצירה לקו

```
TransportResult TransportAddStation(TransportDB* tdb, int line_id, const  
char *station)
```

תיאור הפעולה: הוספת תחנת עצירה לקו. התחנה תתווסף בסוף רשימת התחנות בקו.
פרמטרים:

- <tdb> מצביע למבנה הנתונים
- <line_id> מזהה ייחודי של הקו. מספר גדול ממש מ-0
- <station> שם התחנה אותה יש להוסיף

ערכי החזרה אפשריים:

- [TRANSPORT_INVALID_LINE_NUMBER](#) – במקרה בו מספר הקו אינו תקין
- [TRANSPORT_DOESNT_EXIST](#) – במקרה בו מספר הקו אינו קיים במערכת

5.4.6 הסרת תחנת עצירה מקו

```
TransportResult TransportAddStation(TransportDB* tdb, int line_id, unsigned  
int index)
```

תיאור הפעולה: הסרת תחנת עצירה מקו
פרמטרים:

- <tdb> מצביע למבנה הנתונים
- <line_id> מזהה ייחודי של הקו. מספר גדול ממש מ-0
- <index> מיקום התחנה אותה יש להסיר כאשר התחנה הראשונה נמצאת במיקום 0

ערכי החזרה אפשריים:

- [TRANSPORT_INVALID_LINE_NUMBER](#) – במקרה בו מספר הקו אינו תקין
- [TRANSPORT_DOESNT_EXIST](#) – במקרה בו מספר הקו או התחנה אינו קיים במערכת

```
TransportResult TransportReportLines(TransportDB* tdb, const char *type)
```

תיאור הפעולה: פקודה זו מדפיסה לערוץ הפלט את כל כלי התחבורה הקיימים במערכת מהסוג המבוקש. הקוים יודפסו על פי סדר מספרי. (יתכן ויהיה עליכם למיין את הרשימה לפני ההדפסה או לתחזק רשימה ממויינת). ההדפסה תתבצע באמצעות הפונקציה prog2_report_line המוגדרת בקובץ prog2_ex2.h. עבור כל קו יודפסו הפרטים הבאים: מספר מזהה, סוג הקו, מספר התחנות בקו ועלות הנסיעה בקו

פרמטרים:

- <tdb> מצביע למבנה הנתונים
- <type> מחרוזת המכילה אחד מהבאים בלבד: "BUS", "METRO", "TRAIN", "ALL". כל מחרוזת אחרת אינה חוקית! מערכת תדפיס קווי תחבורה מהסוג המבוקש או את כולם במידה ונשלחה המחרוזת ALL

ערכי החזרה אפשריים:

- [TRANSPORT_INVALID_LINE_TYPE](#) – במקרה בו סוג הקו אינו תקין
- [TRANSPORT_EMPTY](#) – במקרה בו אין כלל קוים במערכת או אין כלל קוים מהסוג המבוקש

```
TransportResult TransportReportStations(TransportDB* tdb, int line_id)
```

תיאור הפעולה: פקודה זו מדפיסה לערוץ הפלט את כל התחנות הקיימות בקו מסוים. ראשית יש להדפיס את פרטי הקו באמצעות הפונקציה prog2_report_line המוגדרת בקובץ prog2_ex1.h ורק אז התחנות יודפסו על פי המיקום שלהן בקו. ההדפסה של התחנות תתבצע באמצעות הפונקציה prog2_report_station המוגדרת בקובץ prog2_ex2.h.

פרמטרים:

- <tdb> מצביע למבנה הנתונים
- <line_id> מזהה ייחודי של הקו. מספר גדול ממש מ-0

ערכי החזרה אפשריים:

- [TRANSPORT_INVALID_LINE_NUMBER](#) – במקרה בו מספר הקו אינו תקין
- [TRANSPORT_DOESNT_EXIST](#) – במקרה בו מספר הקו אינו קיים במערכת
- [TRANSPORT_EMPTY](#) – במקרה בו אין כלל תחנות בקו המבוקש

```
TransportResult TransportReportDirections(TransportDB* tdb, const char
*from, const char *to)
```

תיאור הפעולה: פקודה זו מדפיסה את כל הנסיעות אשר בעזרתן ניתן להגיע מתחנת מוצא לתחנת יעד. כלומר, יודפסו כל הקוים אשר תחנת המוצא מופיעה בהם לפני תחנת היעד (מובן ששתי התחנות נדרשות להופיע במסלול של הקו).
הקוים יודפסו על פי סדר התעריף מהזול ליקר באופן הבא: מספר הקו, סוג הקו, מחיר הקו, תחנה ראשונה ותחנה סופית באמצעות הפונקציות `prog2_report_station` המסופקת לכם גם היא בקובץ `prog2_ex2.h`. במקרה ויש תחנה אחת בקו היא תודפס גם כראשונה וגם כאחרונה.
רמז: ניתן ליצור רשימת קוים זמנית שמסודרת על פי מחיר ולא על פי מספר קו.

פרמטרים:

- `<tdb>` מצביע למבנה הנתונים
- `<from>` תחנת המוצא לחיפוש
- `<to>` תחנת היעד לחיפוש

ערכי החזרה אפשריים:

- `TRANSPORT_DOESNT_EXIST` – במקרה בו אין נסיעה בקו המכיל את שתי התחנות בסדר הנתון
- `TRANSPORT_EMPTY` – במקרה בו אין כלל קוי תחבורה במאגר הנתונים

5.5 הגבלות על המימוש

- עליכם לממש את המערכת תוך התחשבות במגבלות הבאות:
 - אין הגבלה על כמות הקוים או על מספר התחנות בכל קו.
 - שימו לב שעליכם לכלול `#include "prog2_ex2.h"` את הקובץ הזה בקובץ המימוש שלכם על מנת שתוכלו להשתמש בהגדרות שבו.
 - את כל ההדפסות לפלט יש לבצע באמצעות הפונקציות המתאימות המוגדרות ב `prog2_ex1.h`
 - עליכם להקצות את גודל הזיכרון המתאים עבור על מחרוזת שנשמרת בזיכרון. אין להקצות זיכרון מיותר
 - את מימוש מבני הנתונים במערכת יש לבצע באמצעות רשימות מקושרות ולא באמצעות מערכים. הסיבות לכך הן:
 - לאחר שמתרגלים, רשימה מקושרת מתאימה יותר לדרישות התרגיל ומקלה על המימוש.
 - כחלק ממטרת התרגיל, שיפור ביכולת כתיבת מבני נתונים מורכבים ושימוש במצביעים ובהקצאות זיכרון באופן דינמי
 - בכל המקרים, ביציאה מהתוכנית, יש לשחרר באופן מפורש את כל המשאבים שהוקצו למערכת.

5.6 הידור, קישור ובדיקה

על המימוש שלכם לעבור הידור בעזרת הפקודה הבאה:

```
gcc -o transport_db -std=c99 -Werror -pedantic-errors *.c prog2_ex2.o
```

משמעות הפרמטרים :

- transport_db -o זהו שם קובץ ההרצה המהודר
- std=c99 קביעת תקן לשפה
- Werror דווח על כל השגיאות בזמן קומפילציה
- pedantic-errors התייחס לאזהרות כאל שגיאות – משמעות הדגל, שהקוד חייב לעבור הידור ללא אזהרות
- *.c קבצי הקוד אותם נרצה להדר – אלו קבצי הקוד של התרגיל (כולם חייבים להמצא באותה התיקיה)
- prog2_ex2.o קובץ האובייקט המכיל את מימושי פונקציות השירות לשימושכם

התרגיל ייבדק בשני אופנים.

בדיקה אחת כוללת מעבר על הקוד ובדקת את איכות הקוד (אי קיום שכפולי קוד, קוד מבולגן ולא קריא "ספגטי", שימוש בטכניקות קידוד "רעות" וכו').
הבדיקה השנייה כוללת את הידור התוכנית המוגשת והרצתה במגוון בדיקות אוטומטיות. על התוכנית לעבור הידור, לרוץ את הבדיקות בשלמותן ולעבור השוואה של קבצי הפלט עם קבצי פלט מצופים.

על מנת לבדוק את תוכניתכם, יסופקו שני מבדקים כאלו עם הפלט המצופה עבור כל אחד. תוכלו להוריד מהאינטרנט תוכנות השוואה בין קבצים (לדוגמה התוכנה meld או להשתמש בטרמינל בפקודה diff) על מנת להשוות את הפלט שלכם עם הפלט המצופה.
שימו לב! התוכנית שלכם תיבדק בדיקות נוספות. בדיקות אלו מכסות רק חלק בסיסי מהמקרים האפשריים.

בדיקת התרגיל בעצמכם מהווה חלק מהתרגיל!

- כתבו בדיקות נוספות בעצמכם בהתאם למפרט התרגיל
- וודאו את נכונות התוכנית שלכם במקרים כלליים ובמקרי קצה
- מומלץ לחשוב על הבדיקות כבר בזמן כתיבת הפתרון
- העדיפו ליצור מספר רב של בדיקות על פני בדיקה אחת גדולה שיהיה לכם קשה להתמצא היכן הטעויות
- וודאו נכונות הקוד לפני ההגשה גם לאחר שינויים קטנים ולכאורה "לא מזיקים"
- שימו לב לדליפות זיכרון! הם ייבדקו באופן אוטומטי על ידי תוכנה מתאימה (valgrind)
- לשם כתיבת בדיקות חדשות הנכם רשאים להשתמש בכלי בינה מלאכותית

6 הגשת התרגיל

עליכם להגיש את פתרון חלק א בקובץ נפרד (word, pdf)

בחלק ב עליכם להגיש אך ורק את הקובץ/קבצים שכתבתם כקבצי טקסט. בפרט אין להגיש את הקבצים המסופקים prog2_ex2.h ו-prog2_ex2.o.

7 דגשים ורמזים

- אין חסם על מספר קוי התחבורה במאגר הנתונים או על מספר התחנות בכל קו כזה.
- חשבו היטב באילו טיפוסים ראוי להשתמש על מנת לממש את הדרישות. מומלץ מאוד, עוד לפני תחילת כתיבת הקוד לצייר באופן סכמטי את מבני הנתונים והקשרים בין הטיפוסים השונים כפי שראיתם בהרצאות ובתרגולים. זהו שלב ה design של המערכת.
- יש להקפיד על ניהול זיכרון נכון. אין להקצות זיכרון מיותר (למשל, זיכרון באורך `MAX_LEN` עבור כל מחרוזת). כמו כן, **יש להקפיד על שחרור של זיכרון כאשר אין בו צורך יותר**
- יש להקפיד על תכנון נכון של התוכנית וכתובה נכונה בשפת C. בשלב זה של לימודיכם הנכם מסוגלים ונדרשים לתכנן ולכתוב תכנית בסדר גודל הנתון בעצמכם. הקפידו לבצע חלוקה נכונה לפונקציות בהתאם למבנה הלוגי של התוכנית ולא לכתוב פונקציית main אחת גדולה.
- נצפה לראות הפרדה בין פונקציות המממשות לוגיקה מסוימת לאחרות.
- **יש להקפיד לא להוציא פלט אלא על ידי הפונקציות שסופקו לכם.**
- במקרה של הסתבכות עם באג קשה:
 - בודדו את הבאג ושחזרו אותו באמצעות מספר מינימלי ככל הניתן של פעולות. ניתן לעשות זאת על ידי מחיקת חלק מהשורות בקוד ובדיקה אם הוא עדיין מתרחש.
 - ניתן להשתמש בהדפסות בריצת התוכנית שמציגות את מצב הריצה (ערכי משתנים, מצביעים וכו).
- מומלץ להשתמש ברשימה של מצביעים לאובייקטים המאוחסנים במאגר הנתונים

 **בהצלחה!**